

For the Breast Cancer dataset, Apply SVM classifier using the Scikit Learn library.

Report the results and inferences by using 70: 30 split criteria for

1. Different Kernel Functions: Linear Kernel, Polynomial Kernel, and RBF kernel
2. For different Regularization parameter $C=0.1, 1, 100$.

Report the following:

1. Compute the Margin width and support vectors for all the combinations of kernels and regularization parameters
2. Compute the confusion matrix, Accuracy and F1 score.
3. For plotting the data on a screen, Use PCA, reduce the dimension of the data into two. Show the support vectors and also the decision boundary

SVM Classification

Data preparation

```
In [1]: import pandas as pd
import numpy as np
import matplotlib.pyplot as plt
import seaborn as sns

from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder

from sklearn.svm import SVC

from sklearn.metrics import confusion_matrix, accuracy_score, f1_score
from sklearn.decomposition import PCA
```

```
In [2]: df = pd.read_csv('breast-cancer.csv')

df = df.drop(['id'], axis=1)
if 'Unnamed: 32' in df.columns:
    df = df.drop(['Unnamed: 32'], axis=1)

le = LabelEncoder()
y = le.fit_transform(df['diagnosis'])
```

```
X = df.drop(['diagnosis'], axis=1)

scaler = StandardScaler()
X_scaled = scaler.fit_transform(X)

X_train, X_test, y_train, y_test = train_test_split(X_scaled, y, test_size=0.3, random_state=42)
```

```
In [3]: import seaborn as sns

kernels = ['linear', 'poly', 'rbf']
C_values = [0.1, 1, 100]

fig, axes = plt.subplots(3, 3, figsize=(15, 15))
fig.suptitle('Confusion Matrices for SVM (Full 300 Data)', fontsize=18, y=1.05)

print("--- SVM MODEL EVALUATION TEXT REPORT ---\n")

for i, kernel in enumerate(kernels):
    for j, C in enumerate(C_values):
        clf = SVC(kernel=kernel, C=C, random_state=42)
        clf.fit(X_train, y_train)

        y_pred = clf.predict(X_test)

        acc = accuracy_score(y_test, y_pred)
        f1 = f1_score(y_test, y_pred)
        cm = confusion_matrix(y_test, y_pred)

        total_sv = len(clf.support_vectors_)

        if kernel == 'linear':
            w_norm = np.linalg.norm(clf.coef_[0])
            margin_width = f"{2 / w_norm:.4f}"
        else:
            margin_width = "N/A (Non-linear space)"

        print(f"Kernel: {kernel.upper()} | C: {C}")
        print(f"Accuracy: {acc:.4f} | F1 Score: {f1:.4f}")
        print(f"Total Support Vectors: {total_sv} | Margin Width: {margin_width}")
        print("-" * 60)

        ax = axes[i, j]

        sns.heatmap(cm, annot=True, fmt='d', cmap='Blues', ax=ax, cbar=False,
                    annot_kws={"size": 14}, # Make the numbers inside the
                    xticklabels=['Benign (0)', 'Malignant (1)'],
                    yticklabels=['Benign (0)', 'Malignant (1)'])

        ax.set_title(f"Kernel: {kernel} | C: {C}\nAcc: {acc:.2f} | F1: {f1:.2f}")

        if i == 2: ax.set_xlabel('Predicted Diagnosis', fontsize=11)
        if j == 0: ax.set_ylabel('True Diagnosis', fontsize=11)

plt.tight_layout()
plt.subplots_adjust(top=0.90)
plt.show()
```

--- SVM MODEL EVALUATION TEXT REPORT ---

Kernel: LINEAR | C: 0.1
Accuracy: 0.9708 | F1 Score: 0.9558
Total Support Vectors: 52 | Margin Width: 1.3751

Kernel: LINEAR | C: 1
Accuracy: 0.9883 | F1 Score: 0.9828
Total Support Vectors: 32 | Margin Width: 0.7758

Kernel: LINEAR | C: 100
Accuracy: 0.9649 | F1 Score: 0.9483
Total Support Vectors: 29 | Margin Width: 0.0845

Kernel: POLY | C: 0.1
Accuracy: 0.8480 | F1 Score: 0.7174
Total Support Vectors: 204 | Margin Width: N/A (Non-linear space)

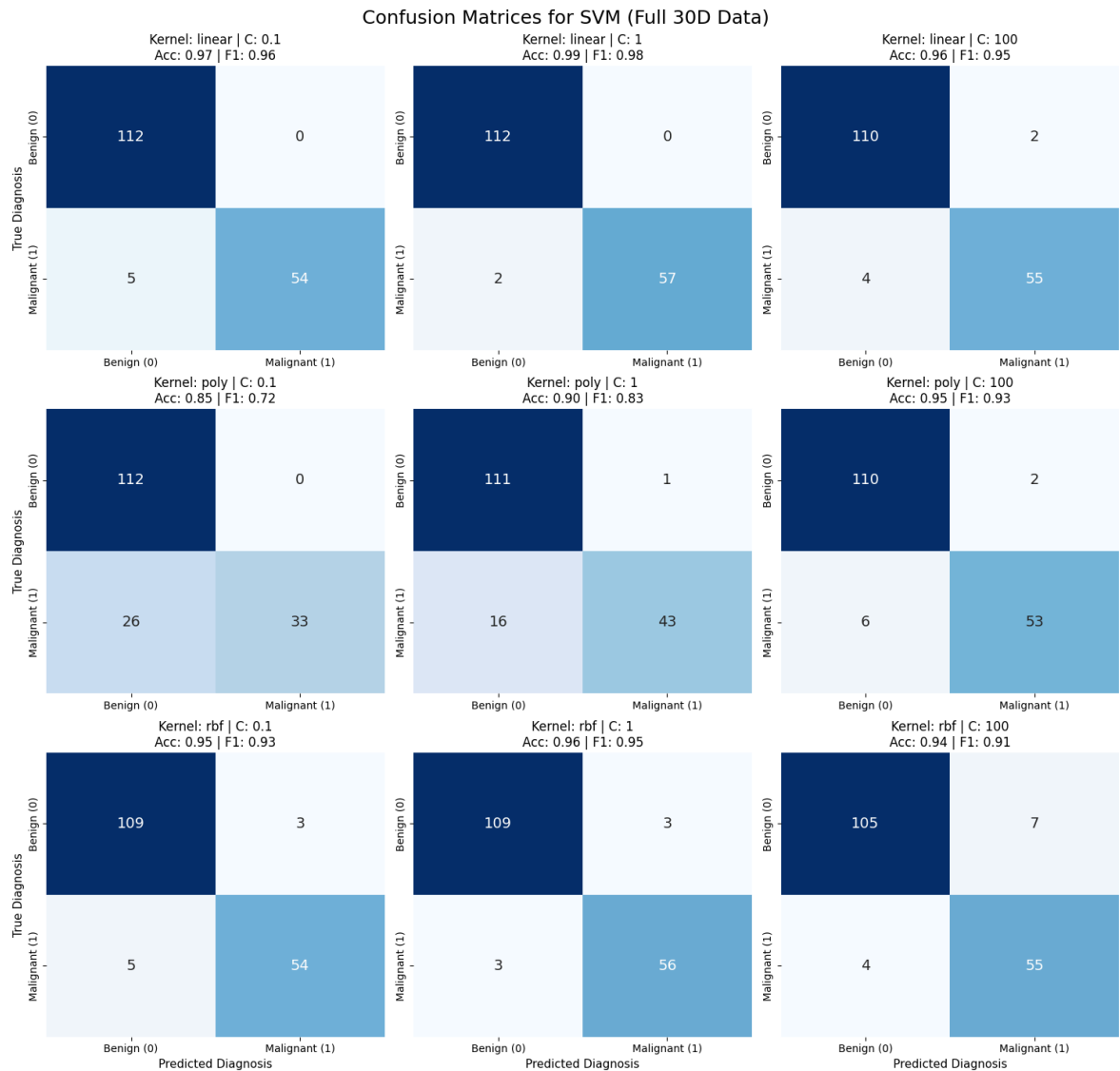
Kernel: POLY | C: 1
Accuracy: 0.9006 | F1 Score: 0.8350
Total Support Vectors: 128 | Margin Width: N/A (Non-linear space)

Kernel: POLY | C: 100
Accuracy: 0.9532 | F1 Score: 0.9298
Total Support Vectors: 63 | Margin Width: N/A (Non-linear space)

Kernel: RBF | C: 0.1
Accuracy: 0.9532 | F1 Score: 0.9310
Total Support Vectors: 187 | Margin Width: N/A (Non-linear space)

Kernel: RBF | C: 1
Accuracy: 0.9649 | F1 Score: 0.9492
Total Support Vectors: 101 | Margin Width: N/A (Non-linear space)

Kernel: RBF | C: 100
Accuracy: 0.9357 | F1 Score: 0.9091
Total Support Vectors: 70 | Margin Width: N/A (Non-linear space)



PCA Application and feature reduction

```
In [4]: # 1. Apply PCA to reduce to 2 dimensions
pca = PCA(n_components=2)
X_pca = pca.fit_transform(X_scaled)

# 2. Split the PCA-reduced data (70:30)
X_train_pca, X_test_pca, y_train_pca, y_test_pca = train_test_split(X_pca, y,
                                                                      test_size=0.3,
                                                                      random_state=42)

# Set up the matplotlib grid
fig, axes = plt.subplots(3, 3, figsize=(18, 15))
fig.suptitle('SVM Decision Boundaries and Support Vectors (PCA 2D)', fontweight='bold')

# Create a meshgrid for plotting boundaries
h = .02 # step size in the mesh
x_min, x_max = X_pca[:, 0].min() - 1, X_pca[:, 0].max() + 1
y_min, y_max = X_pca[:, 1].min() - 1, X_pca[:, 1].max() + 1
xx, yy = np.meshgrid(np.arange(x_min, x_max, h), np.arange(y_min, y_max, h))

for i, kernel in enumerate(kernels):
    for j, C in enumerate(C_values):
        ax = axes[i, j]

        # Train on 2D PCA data
```

```

clf_pca = SVC(kernel=kernel, C=C, random_state=42)
clf_pca.fit(X_train_pca, y_train_pca)

# Predict on meshgrid to create decision boundary
Z = clf_pca.predict(np.c_[xx.ravel(), yy.ravel()])
Z = Z.reshape(xx.shape)

# Plot decision contour
ax.contourf(xx, yy, Z, cmap=plt.cm.coolwarm, alpha=0.3)

# Plot data points
scatter = ax.scatter(X_train_pca[:, 0], X_train_pca[:, 1], c=y_train_pca,
                    cmap=plt.cm.coolwarm, edgecolors='k', s=20)

# Highlight support vectors
sv = clf_pca.support_vectors_
ax.scatter(sv[:, 0], sv[:, 1], s=100, facecolors='none', edgecolor='k',
          linewidths=1.5, label='Support Vectors')

ax.set_title(f"Kernel: {kernel} | C: {C}")
ax.set_xticks(())
ax.set_yticks(())
if i == 2: ax.set_xlabel('Principal Component 1')
if j == 0: ax.set_ylabel('Principal Component 2')

plt.tight_layout()
plt.subplots_adjust(top=0.92)
plt.show()

```

SVM Decision Boundaries and Support Vectors (PCA 2D)

